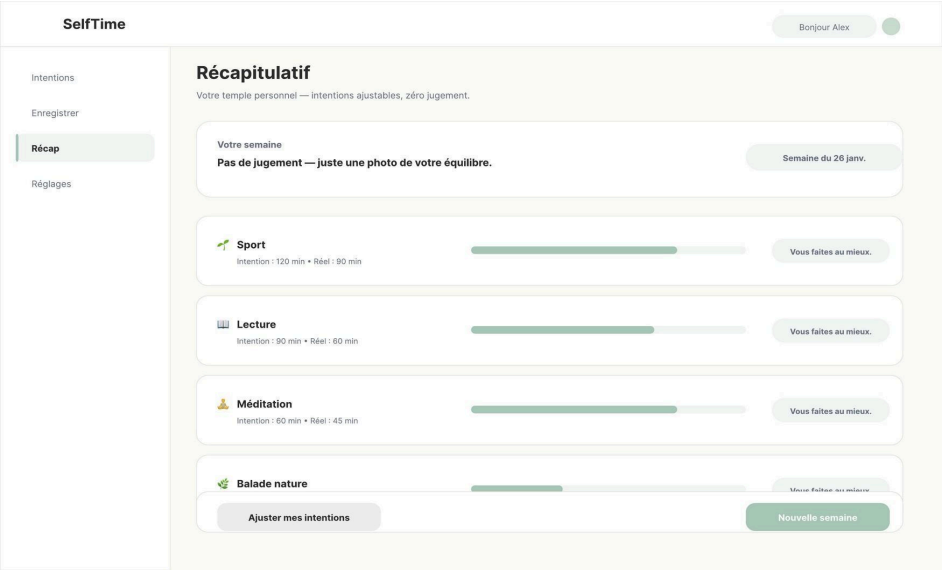
Romain Chazaut

École La Plateforme, 13002 Marseille

2025 / 2026

Dossier de projet SelfTime



Ce dossier est présenté en vue de l'obtention du titre
RNCP 37873 - Concepteur Développeur d'Applications.

1. Introduction

SelfTime, c'est une application web/mobile conçue **pour encourager le bien-être en aidant chacun à prendre du temps pour soi**.

Dans un contexte sociétal, marqué par une prise de conscience de l'importance du bien-être mental et de la gestion saine du temps, de nombreuses personnes souhaiteraient consacrer plus de temps à des activités régénératrices (lecture, sport, méditation, balade), mais peinent à s'y tenir dans la durée. **SelfTime** répond à ce besoin en fournissant un outil qui valorise l'intention plus que la performance et permet d'ajuster ses objectifs en douceur.

Contrairement aux outils de productivité qui mesurent la performance, **SelfTime** se présente comme un espace où l'on définit ses intentions de temps consacrées à des activités bénéfiques et où l'on ajuste ces intentions sans culpabilité : l'interface met en avant des messages comme "intentions ajustables, zéro jugement" et rappelle que l'utilisateur "fait au mieux".

La saisie d'activité est conçue pour être simple et rapide. Il n'y a ni classement ni score : la plateforme insiste sur l'absence de compétition et sur l'importance de se focaliser sur son propre équilibre.

Ce Proof Of Concept a été développé dans le cadre d'un projet pédagogique en 2025 / 2026 pour illustrer une architecture complète (front-end, back-end et base de données).

2. Analyse du besoin et cahier des charges

2.1. Persona et attentes

- **Alex** : actif de 40 ans, souhaitant consacrer du temps à des activités de bien-être (sport, lecture et méditation). Alex cherche un outil pour suivre ses intentions sans pression. Il valorise la confidentialité, la simplicité et un ton bienveillant.

2.2. Exigences fonctionnelles

- **Inscription et connexion :** création d'un compte par e-mail et mot de passe. Les mots de passe sont hachés avant stockage et un jeton JWT est émis après l'authentification.

- **Gestion des activités :**
  - Consulter la liste d'activités pré-enregistrées (sport, lecture, méditation, balade…).
  - Ajouter de nouvelles activités prédéfinies (pour un administrateur).

- **Définition des intentions hebdomadaires :** l'utilisateur choisit une activité et indique une durée cible pour la semaine courante. Plusieurs intentions peuvent être définies, l'application affiche le total hebdomadaire et permet de modifier ou d'ajouter des intentions à tout moment.

- **Enregistrement d'une activité réalisée :** l'utilisateur sélectionne l'activité, saisit la durée (en minutes) et la date, puis valide. Le flux est pensé pour être rapide et accessible depuis la navigation principale.

- **Enregistrement d'une activité réalisée :** affichage, pour chaque activité, du temps réalisé vs. l'intention fixée et une barre de progression simple. Une note bienveillante (« Vous faites au mieux ») accompagne le résumé.

- **Navigation :** barre de navigation en bas de l'écran permettant d'accéder à trois vues (Intentions, Enregistrer (log) et Récap)

2.3. Exigences non-fonctionnelles

- **Simplicité et convivialité :** interface épurée, couleurs apaisantes (nuances de vert), typographie Inter et messages bienveillants.

- **Confidentialité :** aucune donnée ne doit être partagée ou affichée publiquement. Pas de classement ni de score.

- **Sécurité :** API sécurisée avec JWT, stockage des mots de passe hachés, validation des données côté serveur.

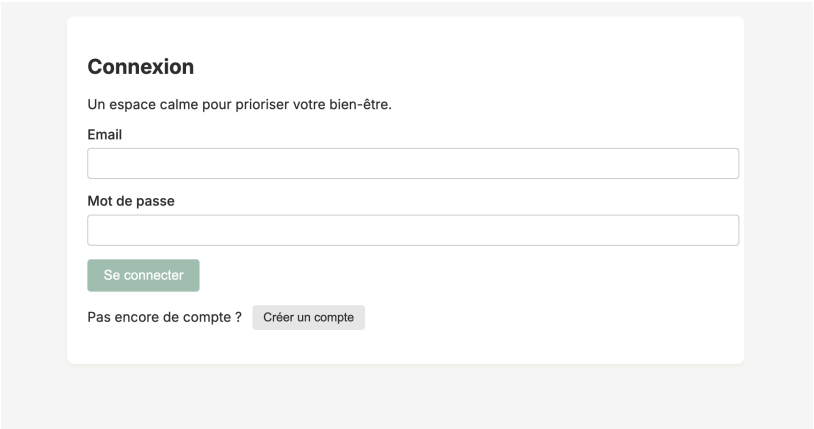
- **Performance :** API REST réactive ; objectif P95 < 200 ms pour les opérations courantes (lecture/écriture de logs et d'objectifs). Utilisation d'index sur les clés primaires et étrangères.

- **Scalabilité :** architecture en couches (controllers → services → repositories) facilitant l'ajout de fonctionnalités et le passage à un environnement micro-services.

3. Parcours utilisateur

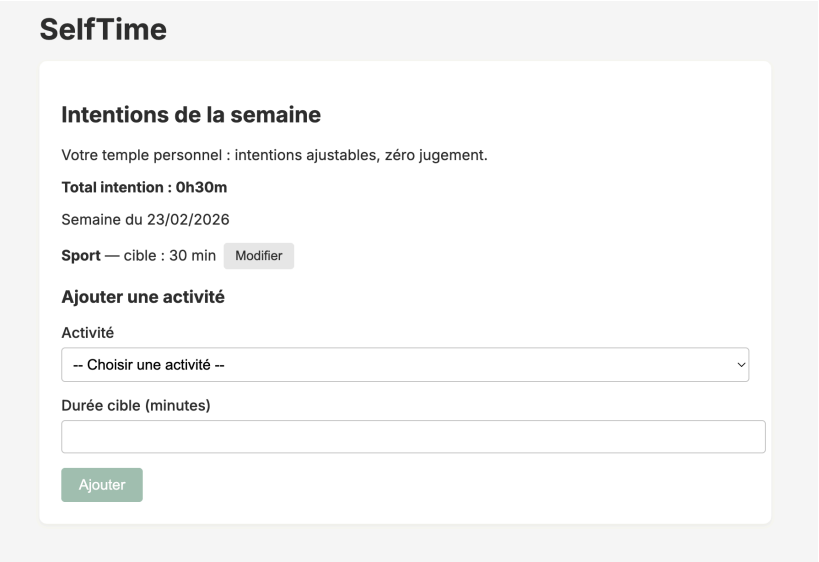
3.1 Onboarding et connexion

L'utilisateur arrive sur une page d'accueil minimaliste, crée un compte ou se connecte avec son e-mail et son mot de passe. Un jeton JWT est stocké localement après la connexion.



3.2 Définition des intentions

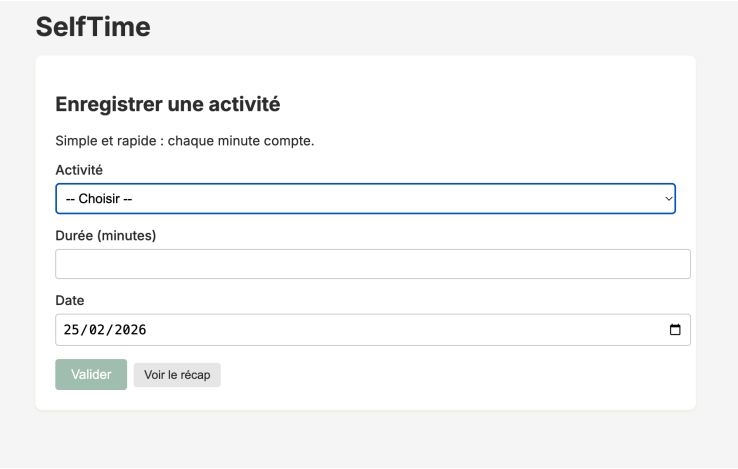
- Depuis l'onglet Intentions, l'application affiche les intentions de la semaine et le total horaire. Alex peut sélectionner une activité existante et définir une durée cible (en minutes), ou modifier une intention existante via le bouton "Modifier". `



- L'utilisateur peut ajouter une nouvelle intention en sélectionnant l'activité et la durée, puis enregistrer. Le total hebdomadaire se met à jour immédiatement.

3.3 Enregistrement d'une activité

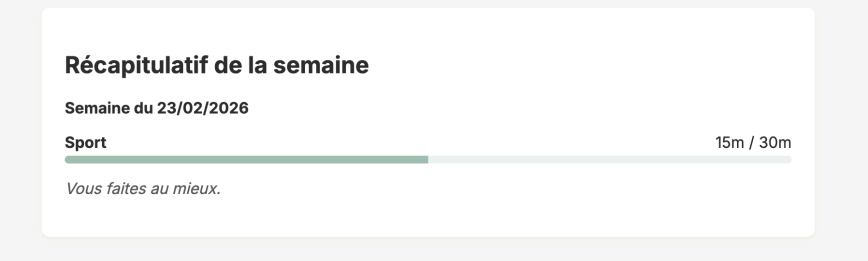
- Dans l'onglet Enregistrer, l'utilisateur choisit une activité, saisit la durée réalisée et la date (par défaut la date du jour) et valide. Un message confirme la prise en compte.



- L'utilisateur peut accéder directement au récap depuis cette page pour visualiser ses progrès.

3.5 Consultation du récapitulatif

- L'onglet Récap affiche pour la semaine en cours la liste des activités avec la durée réalisée et la durée cible, ainsi qu'une barre de progression et un message bienveillant.



- Si aucune intention n'a été définie, un message l'invite à définir ses objectifs.

Une barre de navigation permet de passer d'une vue à l'autre sans perdre le contexte. À tout moment, l'utilisateur peut ajuster ses intentions.

4. Maquettes

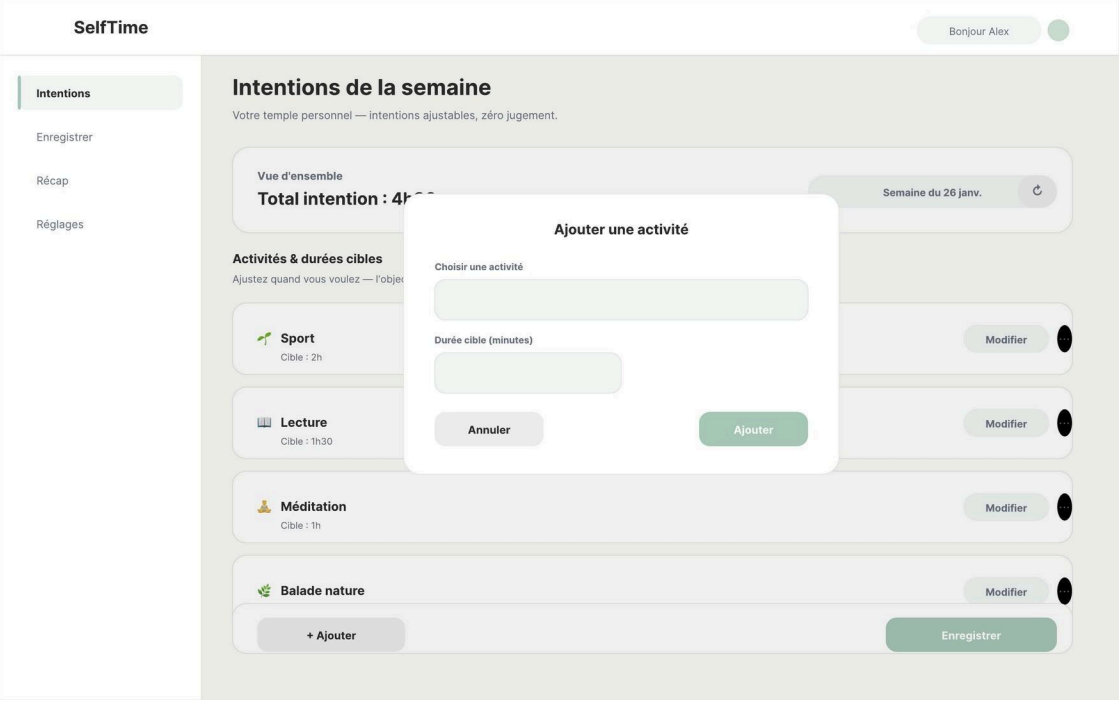
4.1. Écran « Intentions de la semaine »

L'écran suivant (version haute fidélité) présente les intentions hebdomadaires de l'utilisateur.

Pour chaque activité, on voit la durée cible et un bouton Modifier.

Un résumé indique le total hebdomadaire et la date de début de semaine.

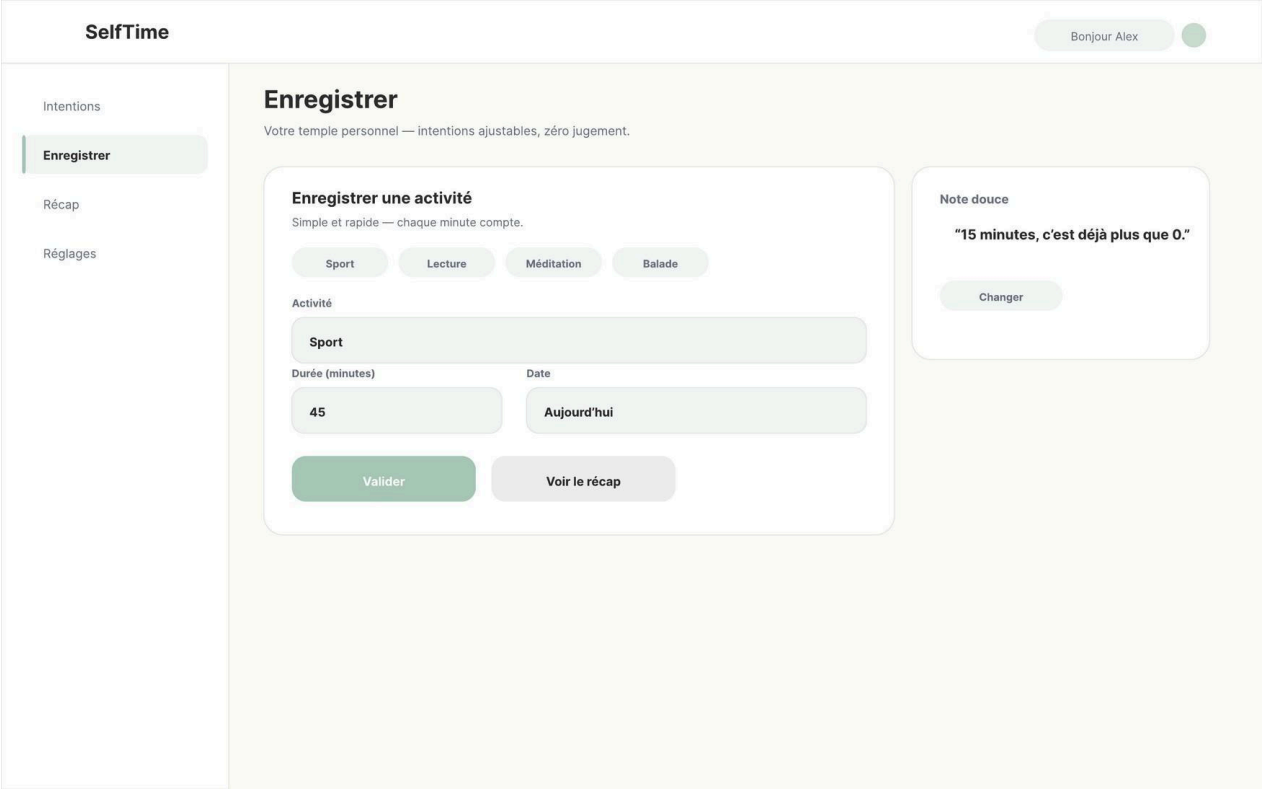
Un bouton permet d'ajouter une activité.



Écran intentions hebdomadaires

4.2. Écran « Enregistrer une activité »

L'utilisateur sélectionne l'activité réalisée, saisit la durée et la date puis valide. L'ergonomie est pensée pour que la saisie prenne quelques secondes, conformément au message "Chaque minute compte".



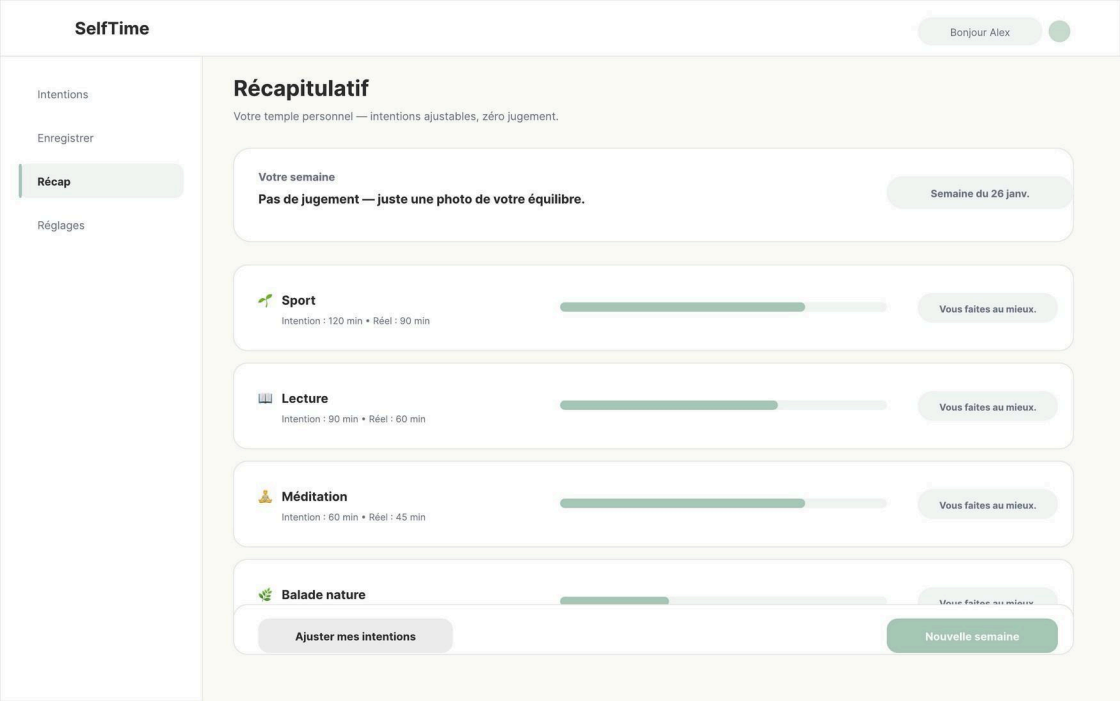
Écran d'enregistrement d'une activité

4.3. Écran « Récapitulatif »

Le récapitulatif hebdomadaire affiche, pour chaque activité, le temps réalisé et l'intention.

Une barre de progression visuelle aide l'utilisateur à voir son équilibre, un message bienveillant rappelle qu'il n'y a pas de jugement.

Le récapitulatif hebdomadaire affiche, pour chaque activité, le temps réalisé et l'intention. Une barre de progression visuelle aide l'utilisateur à voir son équilibre ; un message bienveillant rappelle qu'il n'y a pas de jugement.



Écran récapitulatif

5. Architecture logicielle

L'architecture du POC adopte une approche multicouche inspirée du modèle MVC :

- **Front-end (client) :** application web réalisée avec React + Vite. Elle gère la navigation et la présentation (pages *Intentions*, *Log*, *Récap*) et stocke le jeton JWT dans le stockage local pour authentifier les requêtes.

- **API REST :** serveur Node/Express. Les *controllers* reçoivent les requêtes HTTP, vérifient les paramètres et délèguent la logique à la couche service. Un middleware d'authentification extrait et valide le JWT pour protéger les routes.

- **Services et repositories :** les services implémentent la logique métier (création des intentions, enregistrement des logs, génération du récap) et utilisent des *repositories* pour accéder aux données. Cette séparation facilite les tests unitaires et l'évolution de la logique.

- **Couche de persistance :**
  - PostgreSQL (via Sequelize) stocke les utilisateurs, les activités, les objectifs hebdomadaires et les logs. Les identifiants sont des UUID, et des contraintes d'unicité sont appliquées sur les e-mails et les noms d'activités. Des index favorisent les recherches par utilisateur et par semaine.
  - MongoDB (via Mongoose) conserve les récapitulatifs hebdomadaires. Chaque document associe un utilisateur et une semaine à la liste des activités, aux durées cibles et réalisées, et à une note bienveillante. L'utilisation d'une base NoSQL démontre la possibilité de mixer des technologies de stockage.

```
[→  selftime_project (1) 2 tree -L1
.
├── docker-compose.yml
├── selftime_app
└── selftime_frontend

3 directories, 1 file
[→  selftime_project (1) 2 tree -L2
.
├── docker-compose.yml
├── selftime_app
│   ├── Dockerfile
│   ├── node_modules
│   ├── package-lock.json
│   ├── package.json
│   ├── README.md
│   ├── server.js
│   └── src
└── selftime_frontend
    ├── index.html
    ├── node_modules
    ├── package-lock.json
    ├── package.json
    ├── src
    └── vite.config.js

7 directories, 10 files
```

Le schéma ci-dessous illustre cette architecture :

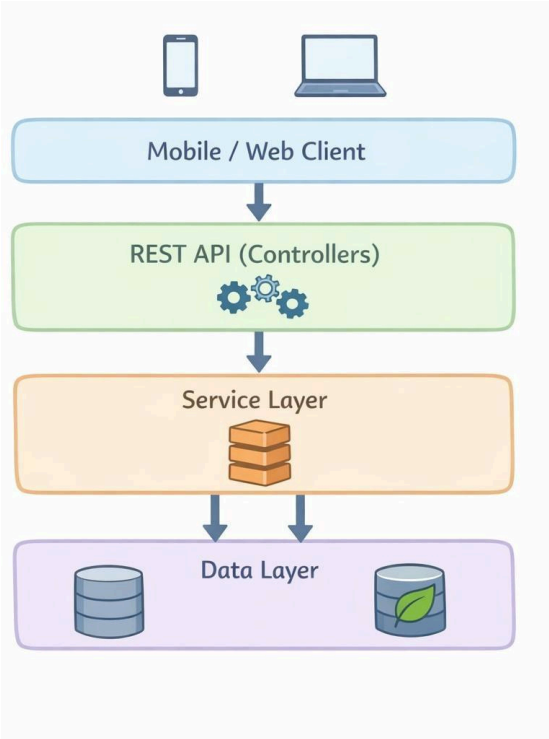


Diagramme d'architecture **SelfTime**

6. Modèle de données

Le modèle conceptuel (MCD) comprend quatre entités principales :

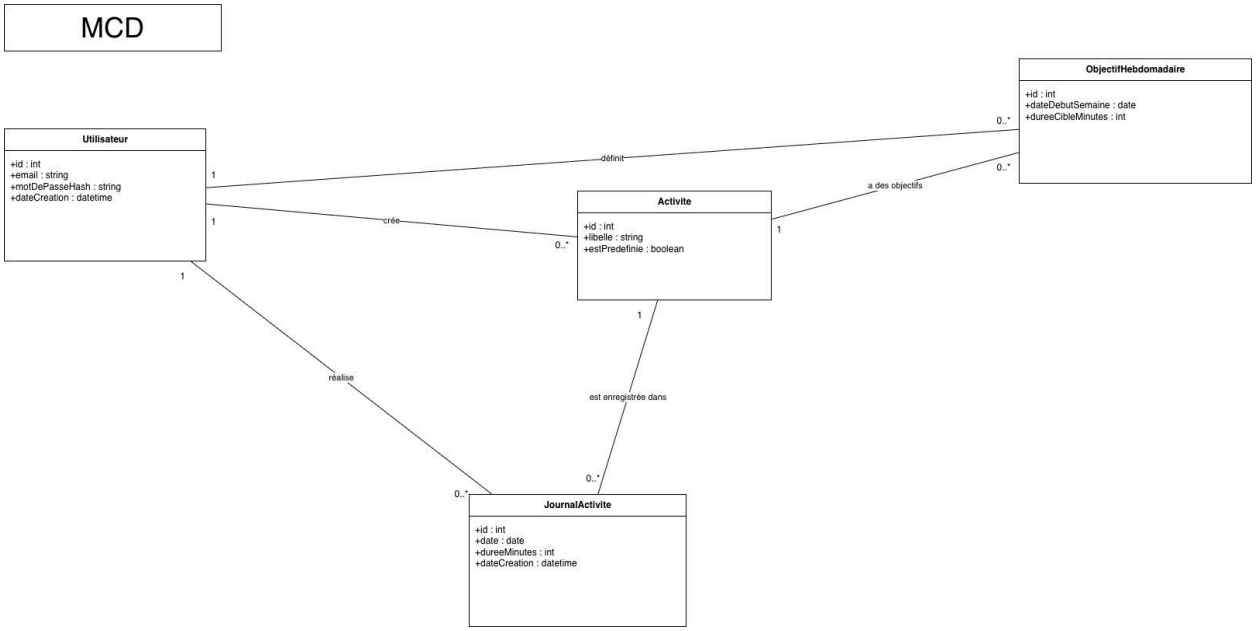
Entité	Attributs clés	Relations
Utilisateur	id, email, passwordHash, createdAt	Définit plusieurs Activités, plusieurs Objectifs hebdomadaires et plusieurs Logs.
Activité	id, name, estPrédefinie	Appartient à un Utilisateur (créateur) et est associée à plusieurs Objectifs hebdomadaires et Logs.
Objectif Hebdomadaire	id, userId, activityId, dureeCible, semaineDebut	Un objectif hebdomadaire pour un utilisateur et une activité donnés.
Journal Activité (Log)	id, userId, activityId, duree, dateActivite, semaineDebut	Historique des activités réalisées.

La conception logique et physique (MLD/MPD) traduit ce modèle en tables SQL :

- users (id UUID PK, email UNIQUE NOT NULL, password NOT NULL, created_at timestamp),

- activities (id UUID PK, name UNIQUE NOT NULL),
- weekly_objectives (id UUID PK, user_id FK → users, activity_id FK → activities,
  duree_cible INT, semaine_debut DATE),
- activity_logs (id UUID PK, user_id FK → users, activity_id FK → activities, duree INT,
  date_activite DATE, semaine_debut DATE),
- recaps (collection MongoDB avec userId, semaineDebut, resumeActivites[] et
  noteBienveillante).

Le schéma entité–relation illustrant les cardinalités est présenté ci-dessous :



Modèle conceptuel (MCD)

7. Choix techniques

- **Langages et frameworks :** JavaScript / TypeScript avec Node.js pour le back-end et React pour le front-end. Ces technologies sont largement utilisées et disposent d'un écosystème riche.

- **Express.js :** facilite la création d'API REST et l'implémentation de middlewares (validation, authentification). Couplé à Sequelize, il permet de gérer la base relationnelle tout en restant flexible.

- **Sécurité :**
  - Hachage des mots de passe avec bcrypt lors de l'inscription ;
  - Authentification stateless grâce à des JSON Web Tokens (JWT) qui encodent l'identifiant utilisateur et expirent après 24 heures ;
  - Middleware auth.middleware.js qui vérifie la présence et la validité du token avant d'autoriser l'accès aux routes protégées.

- **Bases de données :**
  - PostgreSQL pour les entités structurées ;
  - MongoDB pour les récapitulatifs hebdomadaires afin de démontrer l'utilisation d'un stockage NoSQL.

- **Front-end :** Vite pour le bundling rapide, React Router pour la navigation, Axios pour la communication avec l'API. Le JWT est conservé dans localStorage.

- **Containérisation :** un fichier docker-compose.yml permet de lancer simultanément le serveur Node et la base de données dans des conteneurs. Cette approche simplifie le déploiement et la portabilité.

8. Sécurité

Le projet inclut plusieurs mesures de sécurité :

- **Authentification JWT :** les routes sensibles sont protégées par un middleware qui vérifie et décode le jeton. Les tokens expirent après 24 heures et peuvent être révoqués.

- **Hachage des mots de passe :** utilisation de bcrypt avec un salt pour stocker des empreintes non réversibles.

- **Validation des entrées :** contrôles des champs obligatoires et des formats pour éviter les injections SQL ou NoSQL.

- **Gestion des erreurs :** les controllers renvoient des codes HTTP explicites (400 pour les champs manquants, 401 pour l'absence de token, 403 pour les accès non autorisés).

- **Séparation des responsabilités :** la logique métier étant isolée dans la couche service, les contrôleurs ne manipulent pas directement les données, ce qui limite les surfaces d'attaque.

9. Limites du POC

- **Fonctionnalités basiques :** seules les fonctions cœur sont implémentées (gestion des intentions, enregistrement et récapitulatif). Il n'existe pas de dashboard historique par mois ni de notifications.

- **Pas de personnalisation avancée :** la liste des activités est statique dans ce POC et aucun thème visuel n'est proposé à l'utilisateur.

- **Pas d'édition d'utilisateur :** il est impossible de modifier son e-mail, son mot de passe ou de supprimer son compte via l'interface.

- **Absence de déploiement mobile natif :** l'interface est accessible sur le web mais n'est pas empaquetée en application native.

- **Gestion simplifiée des semaines :** la détection de la semaine se base sur le lundi et ne tient pas compte des fuseaux horaires.

- **Pas de test automatisé :** le POC ne contient pas de suite de tests unitaires ou end-to-end ; l'ajout de tests serait indispensable pour un projet de production.

10. Évolutions possibles

- **Personnalisation des activités :** permettre à l'utilisateur de créer ses propres catégories et d'ajouter des icônes pour chaque activité.

- **Notifications et rappels :** envoyer des notifications à l'utilisateur pour lui rappeler ses intentions ou l'inviter à saisir une activité après une période d'inactivité.

- **Historique et statistiques :** fournir des tableaux de bord mensuels ou annuels, avec des graphiques montrant l'évolution du temps consacré à chaque activité.

- **Partage et communauté :** offrir la possibilité de partager ses intentions avec un cercle privé (amis, thérapeute) afin de recevoir du soutien, tout en conservant l'esprit sans compétition.

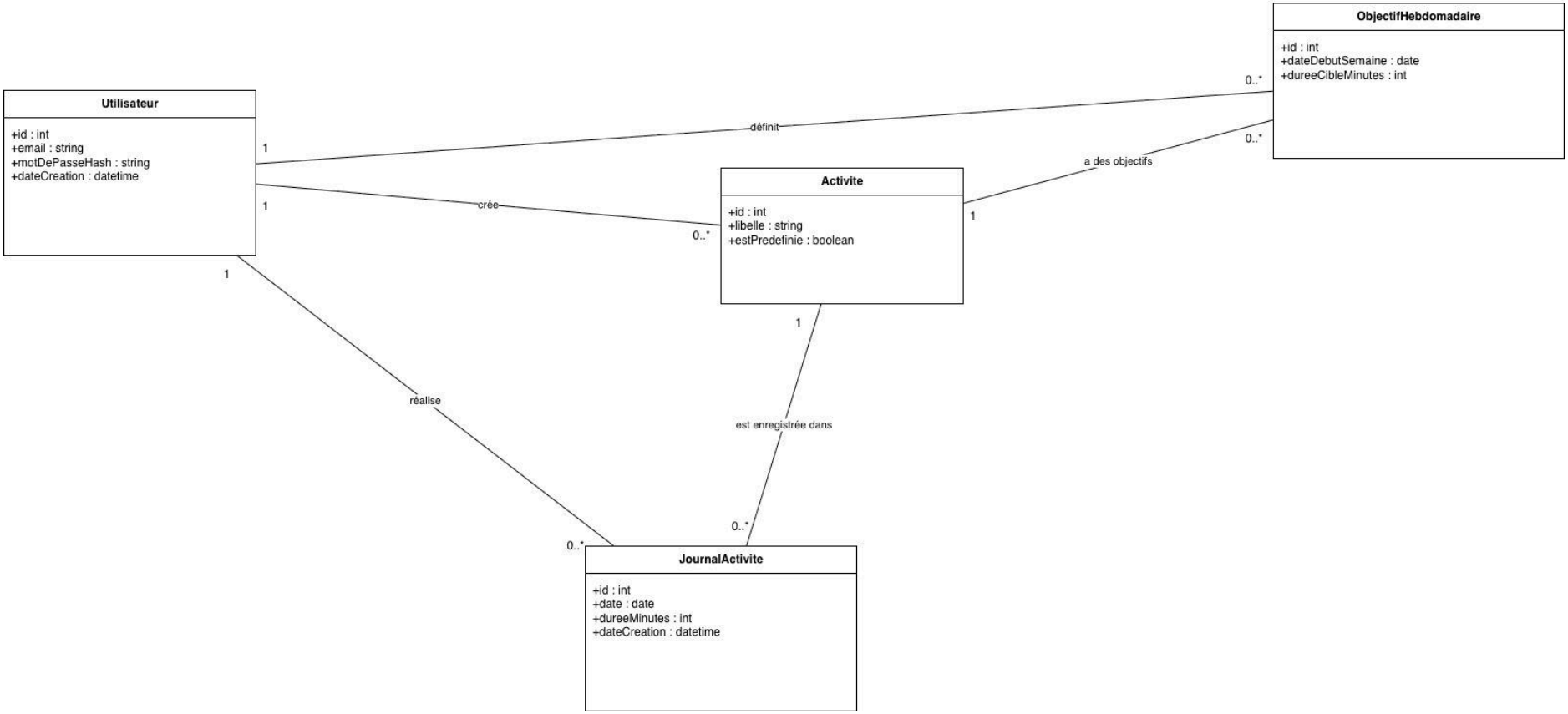
- **Application mobile native :** créer des applications iOS/Android utilisant React Native, permettant l'accès hors ligne et l'intégration avec des capteurs (podomètre, cardiofréquencemètre) pour certaines activités.

- **Intégration de l'IA :** suggérer automatiquement des durées cibles réalistes en fonction de l'historique ou recommander de nouvelles activités bénéfiques.

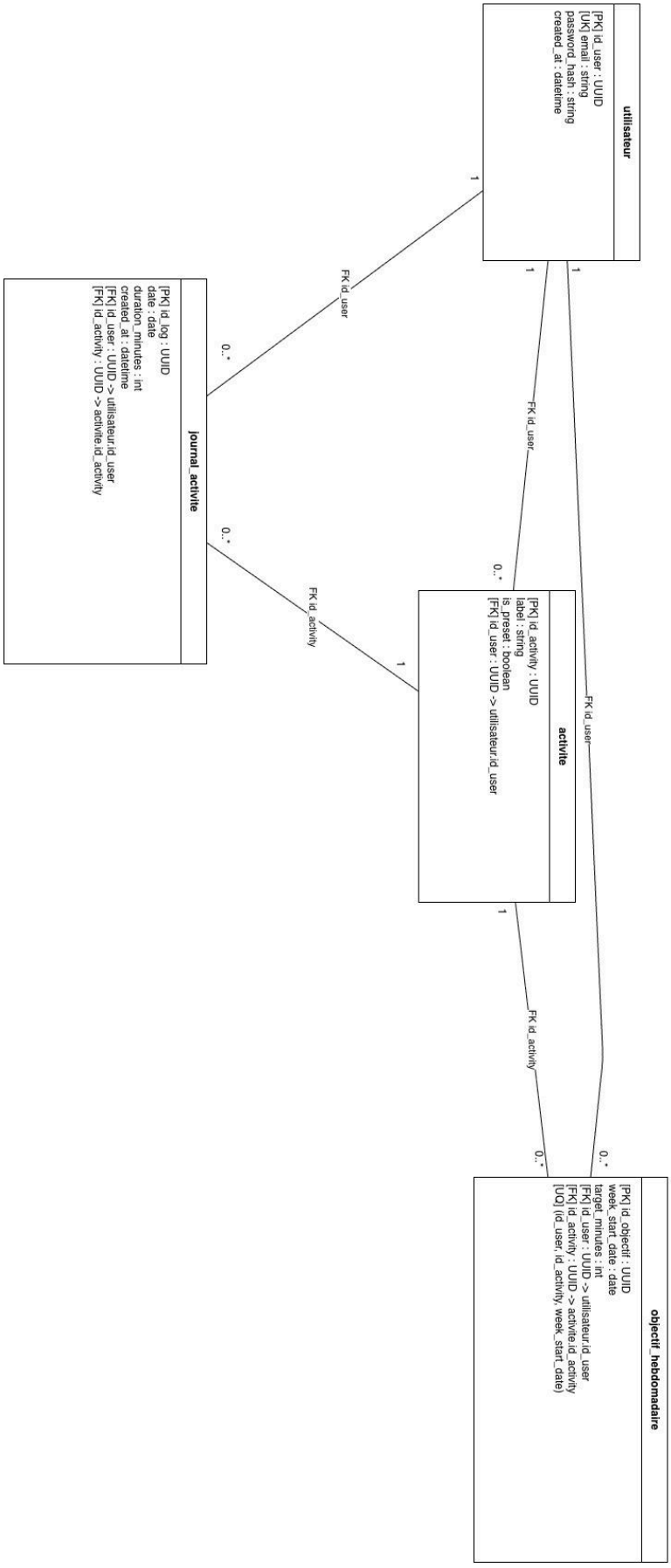
Ce dossier présente la conception et la réalisation du POC **SelfTime**.

Il pourra servir de base pour développer une application complète axée sur le bien-être, respectueuse de la vie privée et conçue pour encourager l'utilisateur à prendre soin de lui-même.

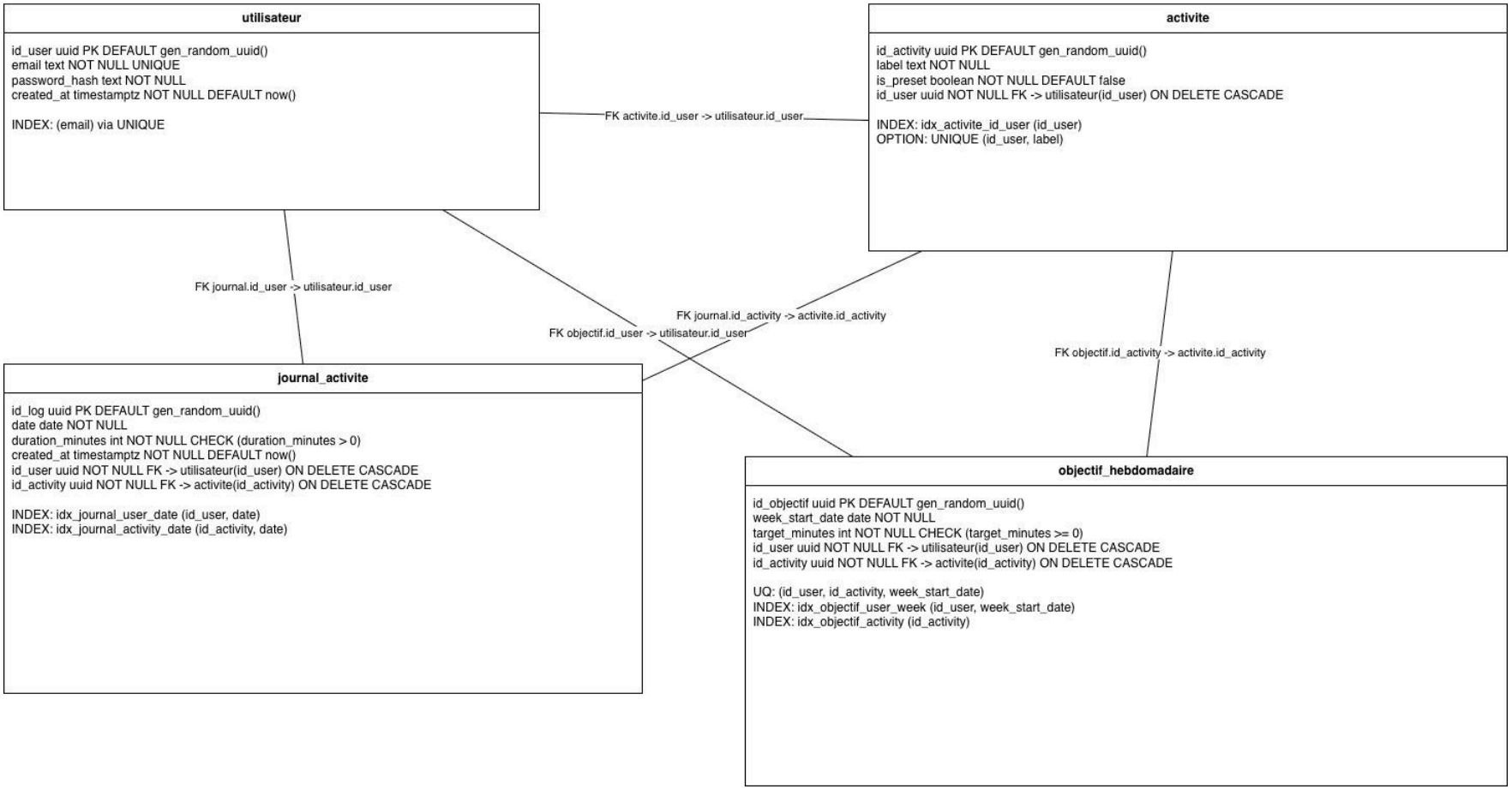
MCD

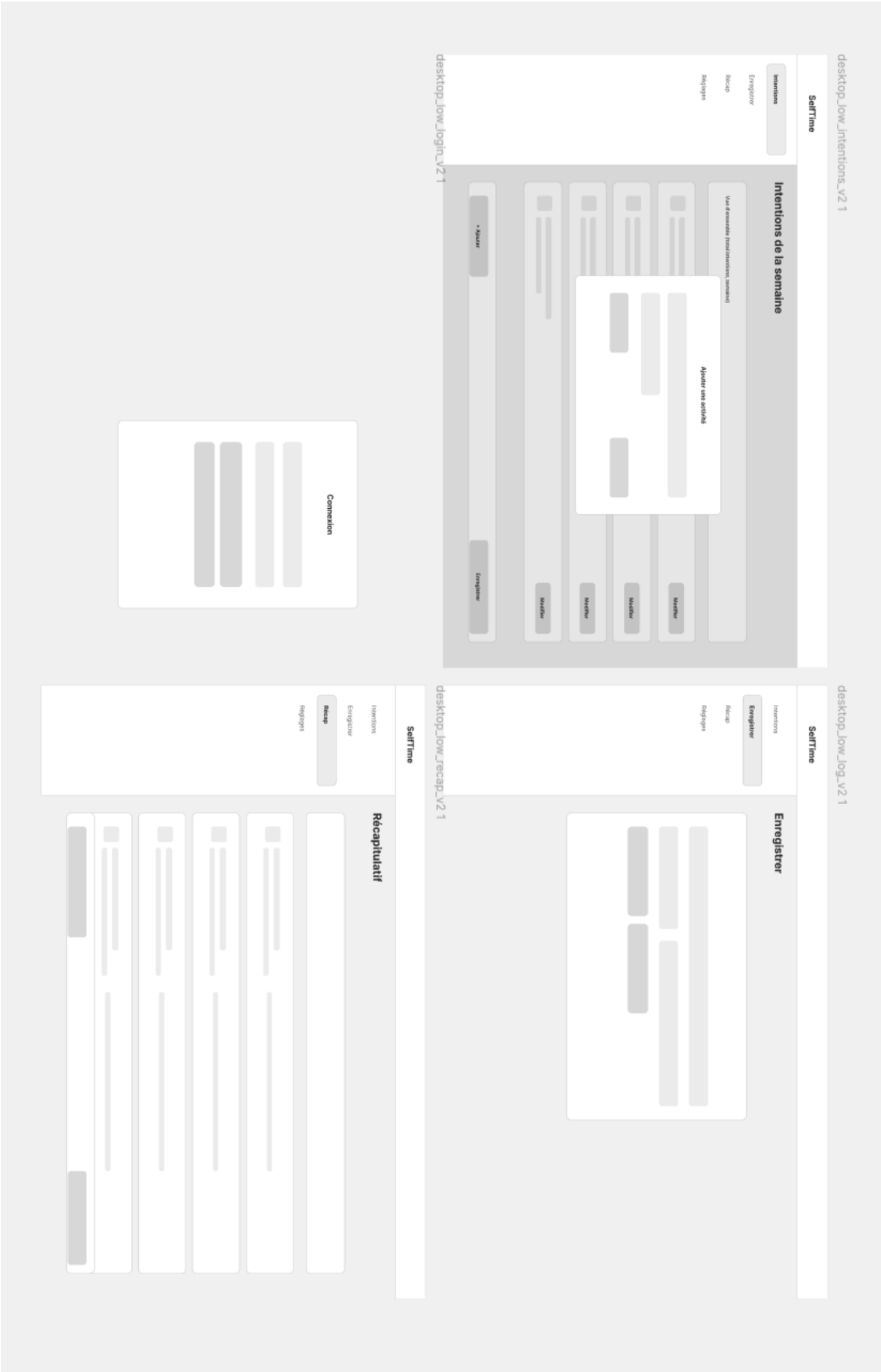

Utilisateur
+id : int
+email : string
+motDePasseHash : string
+dateCreation : datetime
ObjectifHebdomadaire
+id : int
+dateDebutSemaine : date
+dureeCibleMinutes : int
Activite
+id : int
+libelle : string
+estPredefinie : boolean
JournalActivite
+id : int
+date : date
+dureeMinutes : int
+dateCreation : datetime
définit
crée
a des objectifs
réalise
est enregistrée dans
1
0..*

MLD


utilisateur
[PK] id_user : UUID
[UK] email : string
password_hash : string
created_at : datetime
FK id_user
1
0..*
activite
[PK] id_activity : UUID
label : string
is_preset : boolean
[FK] id_user : UUID -> utilisateur.id_user
FK id_activity
1
0..*
objectif_hebdomadaire
[PK] id_objectif : UUID
week_start_date : date
target_minutes : int
[FK] id_user : UUID -> utilisateur.id_user
[FK] id_activity : UUID -> activite.id_activity
[UQ] (id_user, id_activity, week_start_date)
journal_activite
[PK] id_log : UUID
date : date
duration_minutes : int
created_at : datetime
[FK] id_user : UUID -> utilisateur.id_user
[FK] id_activity : UUID -> activite.id_activity

MPD

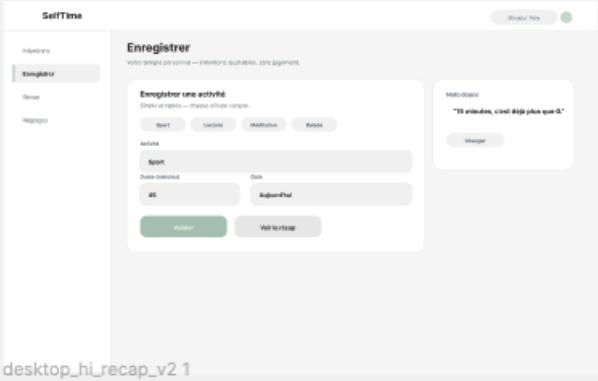

utilisateur
id_user uuid PK DEFAULT gen_random_uuid()
email text NOT NULL UNIQUE
password_hash text NOT NULL
created_at timestamptz NOT NULL DEFAULT now()
INDEX: (email) via UNIQUE
activite
id_activity uuid PK DEFAULT gen_random_uuid()
label text NOT NULL
is_preset boolean NOT NULL DEFAULT false
id_user uuid NOT NULL FK -> utilisateur(id_user) ON DELETE CASCADE
INDEX: idx_activite_id_user (id_user)
OPTION: UNIQUE (id_user, label)
FK activite.id_user -> utilisateur.id_user
FK journal.id_user -> utilisateur.id_user
FK journal.id_activity -> activite.id_activity
FK objectif.id_user -> utilisateur.id_user
FK objectif.id_activity -> activite.id_activity
journal_activite
id_log uuid PK DEFAULT gen_random_uuid()
date date NOT NULL
duration_minutes int NOT NULL CHECK (duration_minutes > 0)
created_at timestamptz NOT NULL DEFAULT now()
id_user uuid NOT NULL FK -> utilisateur(id_user) ON DELETE CASCADE
id_activity uuid NOT NULL FK -> activite(id_activity) ON DELETE CASCADE
INDEX: idx_journal_user_date (id_user, date)
INDEX: idx_journal_activity_date (id_activity, date)
objectif_hebdomadaire
id_objectif uuid PK DEFAULT gen_random_uuid()
week_start_date date NOT NULL
target_minutes int NOT NULL CHECK (target_minutes >= 0)
id_user uuid NOT NULL FK -> utilisateur(id_user) ON DELETE CASCADE
id_activity uuid NOT NULL FK -> activite(id_activity) ON DELETE CASCADE
UQ: (id_user, id_activity, week_start_date)
INDEX: idx_objectif_user_week (id_user, week_start_date)
INDEX: idx_objectif_activity (id_activity)


desktop_low_intentions_v2 1
SelfTime
Intentions
Enregistrer
Récap
Réglages
Intentions de la semaine
Vue d'ensemble (total intentions, semaine)
Ajouter une activité
Modifier
Modifier
Modifier
Modifier
+ Ajouter
Enregistrer
desktop_low_login_v2 1
Connexion
desktop_low_log_v2 1
SelfTime
Intentions
Enregistrer
Récap
Réglages
Enregistrer
desktop_low_recap_v2 1
SelfTime
Intentions
Enregistrer
Récap
Réglages
Récapitulatif

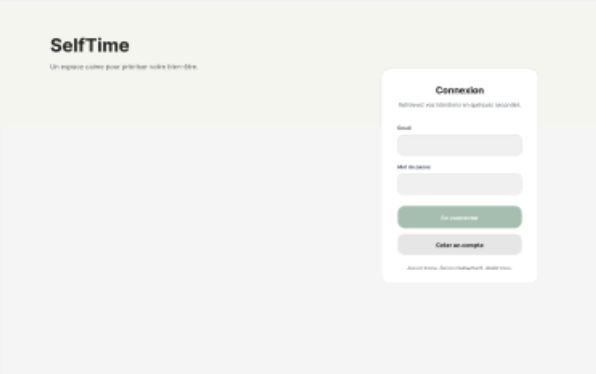
mobile_low_intentions_v2 1
SelfTime
Intentions
Log
Recap
mobile_low_log_v2 1
SelfTime
Intentions
Log
Recap
mobile_low_login_v2 1
mobile_low_recap_v2 1
SelfTime
Intentions
Log
Recap

desktop_hi_intentions_v2 1

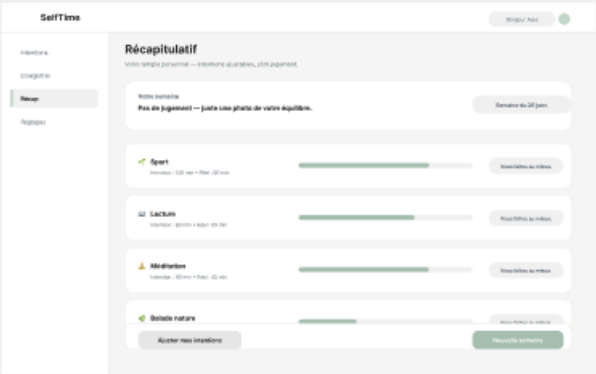
desktop_hi_log_v2 1



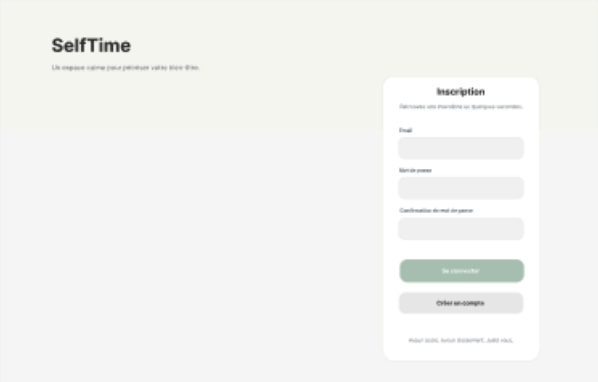
desktop_hi_login_v2 1

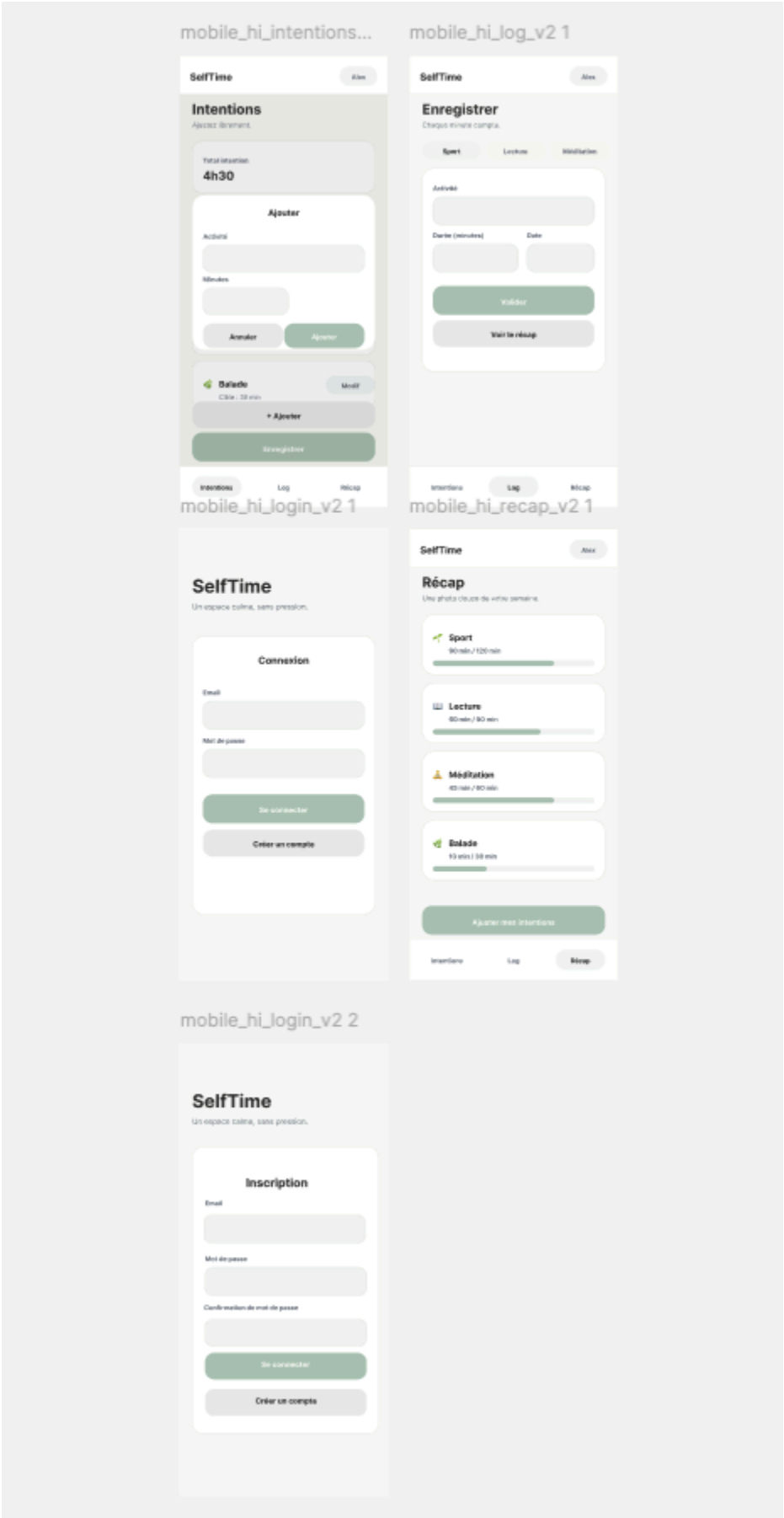


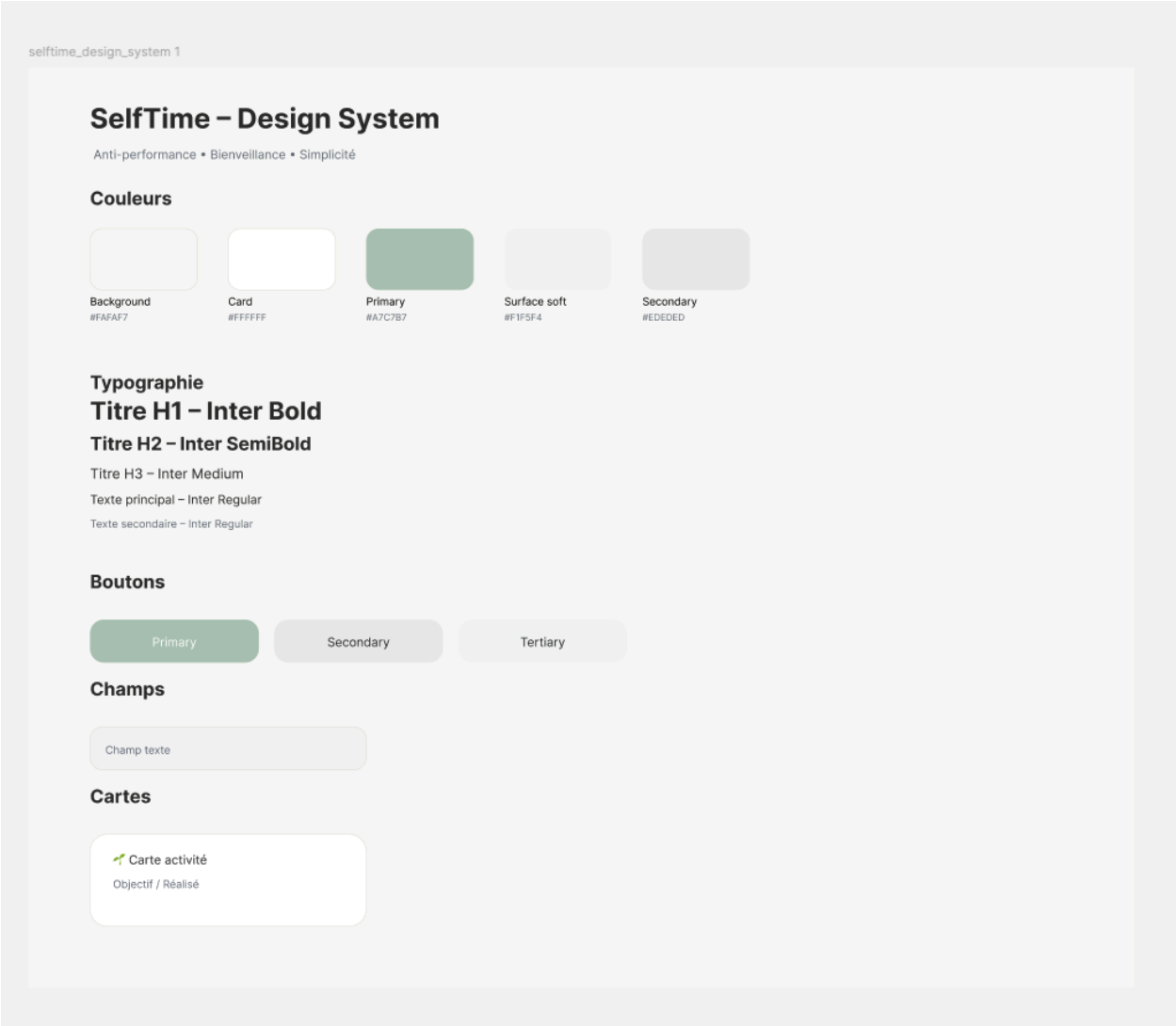
desktop_hi_recap_v2 1



desktop_hi_login_v2 2




mobile_hi_intentions...
SelfTime
Intentions
4h30
Ajouter
Balade
+ Ajouter
mobile_hi_log_v2 1
SelfTime
Enregistrer
mobile_hi_login_v2 1
SelfTime
Connexion
mobile_hi_recap_v2 1
SelfTime
Récap
Sport
Lecture
Méditation
Balade
mobile_hi_login_v2 2
SelfTime
Inscription


selftime_design_system 1
SelfTime – Design System
Anti-performance • Bienveillance • Simplicité
Couleurs
Background
#FAFAF7
Card
#FFFFFF
Primary
#A7C7B7
Surface soft
#F1F5F4
Secondary
#EDEDED
Typographie
Titre H1 – Inter Bold
Titre H2 – Inter SemiBold
Titre H3 – Inter Medium
Texte principal – Inter Regular
Texte secondaire – Inter Regular
Boutons
Primary
Secondary
Tertiary
Champs
Champ texte
Cartes
Carte activité
Objectif / Réalisé